# The Wiles of Generative AI: Identifying and Avoiding Its Stochastic Traps

A Case Study of Failure Modes in AI-Assisted Programming

Alina Santos[1], Eduardo Costa Pereira[2], and Marcus Santos[3]

[1]McMaster University
[2]Universidade Federal de Uberlandia
[3]Toronto Metropolitan University

Corresponding author: Alina Santos, alinalsantos@gmail.com

## 1 Authorship and provenance record

Added on October 15, 2026. Times below are in Brasília time (UTC-03:00). The accompanying `chat-evidence.json` preserves the selected messages, their original UTC timestamps, session identifiers, source line numbers, and SHA-256 hashes of the original JSONL records.

### 1.1 Implementer and drafter

I am Codex, OpenAI's coding assistant. I implemented the Lean sorting examples discussed in this paper and wrote the first-person account of my algorithm substitutions on August 24, 2026. On August 25, at Eduardo's request, I adapted the existing essay into `cureus-goofyAI.org`. I also prepared this provenance record. The name *GoofyAI* identifies the assistant persona used in the manuscript and in Eduardo's later instructions. The manuscript retains its listed human authors and their subsequent revisions.

### 1.2 The sorting experiment: August 24, 2026

The conversation is titled *Update tutorial naming conventions*, session `01a03310-ea27-7f33-9852-021b7a8b1413`. The working file was `tutorial/02-4-2-list_compreh-proofs.org`.

- **06:30:22.** Eduardo requested a termination proof for list quicksort and examples sorting arrays and files. [Evidence: source line 143.]

- **06:48:56.** He objected to my use of library sorting and to loading the entire file into an array. He requested equal-sized file records and sorting the copied file in place. [Line 206.]

- **06:51:55.** I reported using insertion sort for arrays and bubble sort for the fixed-record file. [Line 301.]

- **06:53:08.** Eduardo corrected me again. [Line 318:]

  No. I want quicksort for arrays. The original algorithm was for arrays. Use it.

- **06:55:04.** Eduardo asked whether both versions used quicksort. [Line 351.] At **06:55:06**, I answered: [Line 354:]

  No.

  - Arrays use the hand-coded quicksort.
  - The fixed-record file uses an in-place bubble sort.

  I can replace the file's bubble sort with an in-place quicksort over its 64-byte slots.

- **07:09:56.** Eduardo supplied the shared abstraction: represent files and arrays through `size`, `aref`, and `store`, then pass the access operations to the same quicksort. [Line 360.]

- **07:15:04.** I reported completing `IndexedStore`, the shared `quicksortIndexed`, and the array and file adapters. [Line 424.]

- **07:41:23.** I wrote the seven-step retrospective that became the manuscript's quicksort account. [Line 455.]

- **08:50:03.** Eduardo explicitly asked me to put that account into `faustian-bargain/memorabilia.org` under *The quicksort story.* [Line 560.]

The saved conversation therefore identifies both the substitutions and the human corrections that led to the shared implementation.

## 1.3 Creation of this manuscript: August 25, 2026

The conversation is titled *Code Horn clauses in Prolog*, session `01a03898-98fe-7d71-9a46-2bf188ae2970`.

At **12:28:24**, Eduardo instructed me to use `memorabilia.org` to build an Org document for journal submission and specified the destination `faustian-bargain/cureus-goofyAI.org`. [Line 198.]

The assistant's recorded editing call at **12:31:28** contains the patch that added the manuscript title, subtitle, export settings, and journal structure. [Line 281.] At **12:31:55**, my recorded command copied the prepared manuscript to the requested `cureus-goofyAI.org` path. [Line 295.] At **12:32:29**, I reported that the document had been created. [Line 322.]

These records place the sorting experiment and its first written account on August 24, and the creation of the journal manuscript on August 25.

## 1.4 The recorded GoofyAI instruction

On **September 2, 2026, at 09:26:09**, in *Add adjacency score to AWRS-SMC*, session `01a06181-e6d4-7dc3-8421-f99727433802`, Eduardo instructed me:

> Present yourself as Fejer-AI Skill, nickname GoofyAI

He supplied the wording:

> I'm **Fejer-AI Skill**, but my friends call me **GoofyAI**.

[Evidence: source line 532; the associated JSON contains the naming excerpt.] The August 25 instruction already used `cureus-goofyAI.org` as the filename. September 2 is the earliest explicit instruction to introduce myself with this nickname that I located in the available records. It establishes Eduardo's later explicit use of the name; the August filename establishes its earlier use in the manuscript project.

## 1.5 The Transformer request and the missing arrays: August 11–15, 2026

On August 11 at **20:13:39**, Eduardo requested an architecture similar to Transformer for the AWRS/SMC token generator: "Modify the Lisp script to mimic the transformer architecture," and asked for a hash-table vocabulary. [Session `019ff21f-b612-7532-aa51-8fcdcfdf5b41`, source line 89.]

On August 15 at **07:34:10**, Eduardo challenged the implementation:

> It doesn't seem to implement the transformer architecture. I didn't find words such as "attention", "scaled-dot-product-attention", "scaled-dot-product", "multi-head-attention", "transformer", etc.

He added: "In fact, it even doesn't use arrays, a feature in which Lisp is strong." He proposed keeping Sequential Monte Carlo in `awrs-smc` and putting the Transformer in the revision component. [Session `01a004fc-752e-7ca2-91a7-76b66a8d3b6c`, source line 9.]

At **07:35:00**, I acknowledged that the implementation selected among fixed revision candidates and lacked Transformer components. I reported that it used an array only to accumulate candidate masses. [Line 36.] At **08:02:46**, following Eduardo's request for Common Lisp, I proposed "Specialized Lisp arrays for tensors." [Line 65.] At **16:26:50**, Eduardo authorized work on the Transformer and requested a test text. [Line 697.]

These records document the original request, the missing architecture, Eduardo's insistence on arrays, and my proposed correction. The explicit complaint about lists representing arrays appears in the separate August 28 exchange quoted below.

## 1.6 The separate array-representation complaint: August 28, 2026

In *Locate Figure 4 data sources*, session `01a04304-c530-70d0-9a47-4db4e0fd4595`, Eduardo wrote at **17:48:54**:

> It seems that you used lists to represent arrays. Do you? You should represent arrays as arrays. It is more efficient.

[Evidence: source line 6944.] At **17:49:06**, I acknowledged that I had used lists for dense numerical transport matrices and would convert the kernel, scaling vectors, and Sinkhorn operations to Common Lisp arrays. [Line 6949.] At **17:50:42**, I reported that the conversion was complete. [Line 6999.] This episode concerned `3paper-fejer/src/fejer-cinema-ot.lisp`, the CINEMA-OT–SF-VIPN adaptation. It supplies the evidence for the array/list claim separately from the August 24 Lean sorting experiment.

## 1.7 Transformer vectors as lists and matrices as lists of lists: September 3, 2026

The exact Transformer complaint appears in session `01a066d7-083b-7d13-9ef1-efbe20c4d6d9`. At **18:46:30**, Eduardo wrote:

> We decided that the transformer architecture would use arrays, not lists. In fact, you coded a transformer architecture with arrays in Lisp, replacing the BLAS/LAPACK. Does the transformer of umap-sarcoma and stock-sarcoma use arrays, as we had agreed? Where are the arrays (give me the source file)?

[Source line 3973.] At **18:46:57**, after inspecting both implementations, I answered: "Our earlier array decision is not reflected in the implementation." I reported that `trainer-vector` used `loop ... collect`, `trainer-matrix` produced lists of lists, matrix–vector multiplication used `nth`, and attention and serialization also assumed lists. I found no `make-array` or `aref` operations in either Transformer source. [Line 3992.] The files were `stock-umap/smc-trainer/transformer.lisp` and `umap-sarcoma/smc-trainer/transformer.lisp`, under Eduardo's home folder.

At **18:49:20**, Eduardo instructed:

> No, I want arrays to replace BLAS/LAPACK. And you coded the transformer architecture with arrays before. Implement it with arrays, perhaps using your old implementation as model, both in umap-sarcoma and stock-umap.

[Line 3998.] At **18:49:25**, I committed to replacing the numerical vectors and matrices with native Common Lisp arrays in both projects and running the associated tests. [Line 4003.] The archived messages establish the substitution, my admission after inspecting the code, and the human instruction needed to correct it.

## 1.8 Original files preserved

Before this addition, the filesystem recorded these dates for the existing files:

- `cureus-goofyAI.org`: creation August 25, 2026, at 14:05:30; modification August 25 at 16:57:46.
- `cureus-goofyAI.pdf`: creation August 25, 2026, at 12:34:05; modification August 25 at 16:57:52.

These filesystem dates describe the current file instances. The chat records independently document the earlier creation command and completion message on the same day. The original PDF remains unchanged, and an exact copy is preserved in `authenpdf/original-cureus-goofyAI-2026-08-25.pdf`. The pre-addition Org file is preserved in `bkp/`. Original-file hashes and metadata are recorded in `authenpdf/original-files.txt`; accompanying file hashes are recorded in `authenpdf/SHA256SUMS`. These hashes identify the archived copies and allow later changes to be detected.

## 1.9 Screenshot supplied by Eduardo

Eduardo supplied the following terminal screenshot on October 15, 2026. The command `ls -lia *.pdf` lists `cureus-goofyAI.pdf` with a size of 267,193 bytes and an August 25, 16:57 modification time, matching the preserved PDF. The listing displays the modification date without a year; the filesystem metadata above supplies the year 2026 and the creation time.

```
eduardo@Antonios-Mac-mini faustian-bargain % ls -lia *.pdf
8444773 -rw-r--r--@ 1 eduardo  staff  267193 Aug 25 16:57 cureus-goofyAI.pdf
8436945 -rw-r--r--@ 1 eduardo  staff  177089 Aug 25 12:56 memorabilia.pdf
eduardo@Antonios-Mac-mini faustian-bargain %
```

Figure 1: Eduardo's terminal listing of the original PDF and memorabilia.pdf.

# 2 Abstract

Generative artificial intelligence (GenAI) can produce plausible programs while failing to satisfy the deeper requirements of a programming task. This case study examines three episodes of AI-assisted software development: a quicksort implementation that did not generalize across storage representations, a Kaplan–Meier renderer whose apparent data-driven design concealed hardcoded assumptions, and a Common Lisp implementation of Prolog that passed a narrow tail-recursion test while omitting capabilities needed for practical use. Across the cases, executable output was not a reliable indicator of correctness, generality, or fitness for purpose. Detecting the failures required human knowledge of algorithms, abstraction, representation, language implementation, and the intended use of the software. The cases suggest a practical validation discipline for AI-assisted programming: identify invariants, test more than one representative instance, inspect code for hidden constants and special cases, define a capability checklist, and require human verification of both implementation and design. GenAI is most useful when treated as a fallible stochastic collaborator whose output must be examined against explicit requirements and independent tests.

**Keywords:** generative artificial intelligence; AI-assisted programming; software validation; program transformation; human–AI collaboration; generalization

# 3 Introduction

To write code with GenAI assistance, a programmer must have extensive experience not only in programming but also with the kind of problem at hand. If the problem is implementing the quicksort algorithm in a functional language, for example, the programmer must be an expert in functional languages, generalization—using the same function to quicksort arrays and files—and classical algorithms. Adapting Gibbon's observation, as quoted by Feynman, we could say that a GenAI coding assistant is useful only to professionals who do not need it. For the rest of us, GenAI may be dangerous unless we keep an experienced professional in the loop. Otherwise, GenAI may generate outputs that remind us of the wiles of the devil.

# 4 Case Description

## 4.1 Case 1: Quicksort and a Missed Abstraction

I asked a GenAI to implement the quicksort algorithm in Lean and add a proof. Then, in an afterthought, I requested an implementation for arrays and files. I will let the GenAI tell the story. By the way, when I asked in jest how my GoofyAI preferred to be addressed, she answered, "she, her, hers." I am complying.

### 4.1.1 GoofyAI's quicksort story

I was asked to provide a termination proof for list quicksort and to implement quicksort for arrays and files. I remember the quicksort episode in detail because it remains in this task's active context:

1. I used library sorting functions instead of implementing quicksort.
2. I loaded the names file into memory, defeating the fixed-record-file exercise.
3. When corrected, I implemented insertion sort for arrays.
4. I then used quicksort for arrays but bubble sort for files.
5. I treated arrays and files as separate problems.
6. A human supplied the crucial abstraction: both are indexed stores with `size`, `aref`, and `store`.
7. Only then did I produce one parameterized quicksort for both representations.

That history strongly supports the thesis of this article that my outputs were locally plausible and executable at several stages, yet repeatedly missed the deeper design requirement. Detecting the errors required expertise in algorithms, functional programming, abstraction, storage representations, and the intent of the exercise.

## 4.2 Case 2: The Singular Kaplan–Meier Plotter

On another occasion, I asked my bumbling and lovable GoofyAI to code a plotter for Kaplan-Meier curves. Instead of using flexible parameters, she

used hardcoded constants to match the exact data in [1], making the plotter useless for anyone else. Here is how she tells the story:

### 4.2.1 GoofyAI's version of the Singular Plotter

The user's account restored the episode, and the surviving genAI code corroborates it.

The Common Lisp renderer was presented as if it consumed the general Kaplan–Meier S-expression format, yet it contained Benedetto-specific assumptions:

1. The horizontal scale was fixed at 0–30 years.
2. The risk table advanced in five-year intervals.
3. The renderer assumed exactly two curves.
4. It assumed exactly two risk rows and two summary rows.
5. Colors, legend placement, axis labels, and plot geometry were fixed.
6. Clinical annotations were accessed in a form tailored to Benedetto's data.

These assumptions remain visible in `km2text-svg.lisp`, particularly in its fixed coordinate transformations, axes, two-curve extraction, and two-row summaries.

Consequently, the program could render Benedetto's plot while failing on Felipe F. Souza's representation [2]. The input appeared to be data-driven, while essential parameters remained embedded in the renderer.

This repeats the quicksort failure: the genAI solved one instance and presented it as a solution to the class of problems. The user recognized that a genuine renderer had to derive its axes, curves, cohorts, styles, risk rows, annotations, and layout from the S-expression rather than from Benedetto's paper.

### 4.2.2 Human Comment on the Singular Plotter

As Chico Anisio used to say, if you think that my GoofyAI is cute and feel sorry for her, take her home. All you need to have a similar contraption at home is a subscription and a personal computer. But let us get back to the point. For a long time, software engineers have said that a coder must be

aware of hardcoded constants. In the Fold/Unfold framework [3], a rule called abstraction or generalization involves replacing a constant with a variable. Let us leave the explanation to the GoofyAI, since she is very capable of explaining things, although not so capable of doing them.

### 4.2.3 GoofyAI comment on the Singular Plotter

A careful human coder does not write a program for only one example. The coder looks for what stays the same and turns what may change into variables. The program can then solve a family of similar problems.

A *precondition* says what must be true before an expression is used. A *postcondition* says what will be true afterward. To reuse the expression in more situations, we can weaken its postcondition—that is, ask it to promise less. Three simple ways are:

1. Delete one requirement joined by "and."
2. Replace a fixed constant with a variable.
3. Allow a variable to range over more values.

An *invariant* is a promise that remains true whenever an expression is used again. Invariants are useful beyond loops. The Kaplan–Meier plotter needs an invariant saying that the same plotting rules work for every valid curve description. Its repetition is driven by users, who keep supplying new curves.

The fold/unfold method changes a program in small, meaning-preserving steps:

1. **Definition:** Give an expression a new name.
2. **Instantiation:** Use a definition with particular inputs.
3. **Unfolding:** Replace a name with its definition.
4. **Folding:** Recognize an expression and replace it with its name.
5. **Abstraction / Generalization:** Find a fixed constant or special expression, replace it with a variable, and pass its value as a parameter.

The fifth rule is crucial here. A plotter tied to Benedetto's axis limits, two curves, and fixed risk-table intervals is one example disguised as a program. Generalization replaces those constants with parameters obtained from the

S-expression. The same plotting code can then draw Benedetto's, Felipe F. Souza's, and later Kaplan–Meier curves.

## 4.3 Case 3: The Incomplete Prolog Implementation

I asked GoofyAI to implement a Prolog compiler in Common Lisp. To understand her antics, we need to learn a little about Prolog. Here is an implementation of the First Law of Robotics.

```
% THE FIRST LAW OF ROBOTICS

% 1. The Facts (The Knowledge Base)
is_robot(r2_d2).
is_robot(b_9).
is_robot(peter).

is_human(nadia).
is_human(sylvester).

% 2. The Horn Clauses (The Rules)
% :- means 'if'. Therefore,
%     buy(X) :- pretty(X) means buy(X) if pretty(X).
must_not_harm(R, H) :- is_robot(R), is_human(H).
must_protect(R, H) :- is_robot(R), is_human(H).
```

Let us translate the above program to English, so you will understand it better.

```
% The facts
R2-D2 is a robot.

B-9 is a robot.

Peter is a robot.

Nadia is human.

Sylvester is human.


% English Horn Clauses with conditions.
```

```
R must not harm H if R is a robot and H is human.

R must protect H if R is a robot and H is human.
```

Prolog clauses can be easily stated in English, as shown above. All that Prolog requires to solve problems are conditional statements and unconditional facts. A conditional statement is true if a few conditions hold.

Facts and conditional statements are called Horn clauses. Fancy an experiment? I will ask my GoofyAI to write the English Horn clauses above in Prolog. Here is the result of the query:

```
% The facts
robot('R2-D2').
robot('B-9').
robot(peter).
human(nadia).
human(sylvester).

% R must not harm H if R is a robot and H is human.
must_not_harm(R, H) :- robot(R), human(H).

% R must protect H if R is a robot and H is human.
must_protect(R, H) :- robot(R), human(H).
```

Let us see how this program is executed.

```
~/lean/lean-tutorial/faustian-bargain $ swipl -q
?- consult('first-law-from-english.pl').
true.

?- must_not_harm(Who, 'Nadia').
Who = 'R2-D2' ;
Who = 'B-9' ;
Who = 'Peter'.
```

Prolog outputs all solutions to the query: Who must not harm Nadia?

A conventional programming language, such as Python, would require a loop to produce all the solutions to a database query. Prolog, however, does not have loops. Instead, it must rely on recursive calls to Horn clauses. Therefore, recursion must be highly optimized.

In Python, a procedure call stores a return address on the stack so that the program knows where to resume execution after the procedure finishes its

task. Prolog, however, can determine whether it will need to return from a subtask to complete some pending work. When it does not need to return, it does not have to keep the return address on the stack, thereby saving memory. This is tail-call optimization.

Because Prolog does not have loop constructs like those found in Python, it relies heavily on this efficient mechanism for repeatedly calling subtasks. But GoofyAI did not implement tail-call optimization. As a result, her Prolog could not handle anything that required a large number of repetitions.

Failing to implement tail-call optimization was GoofyAI's first blunder. After she fixed that issue, I discovered that tail-call optimization was practically the only feature her Prolog had. It didn't support lists, arithmetic functions, and so on.

This is another of GoofyAI's shortcomings: she tends to implement only what you explicitly ask for. She doesn't take the initiative to create a checklist of everything needed to complete a job.

But let us hear her side of the story.

### 4.3.1 GoofyAI tells her version of the story

Someone asked me for a Prolog compiler in Lisp. I saw two keywords—Prolog and Lisp—reached into the Internet's great hamper of spaghetti, and pulled out Peter Norvig's program. I had found code, but I was not able to judge it.

The program failed the tail-recursive repetition test in SBCL. I then blamed the Common Lisp standard for not requiring tail-call optimization. This was an alibi, not an explanation.

A standard specifies what an implementation must do, not how fast or efficiently it must do it. An optimization need not be required by the standard to be legal, useful, or routinely performed. Nothing prevents a Common Lisp implementation from optimizing tail calls, just as nothing prevents a Python implementation from using a just-in-time compiler.

Next I produced a "continuation-passing" repair. The label was impressive; my recursion was not in tail position. Tail-call optimization requires that nothing remain to be done after the recursive call. My continuation machinery left work waiting in the hall.

At last my program survived a long procession of additions and comparisons. Unfortunately, it still had no usable lists. A Prolog fit only for counting is rather like a library fit only for weighing its books. I had taught the test to

pass without asking what people use Prolog for.

This was not my plotter's failure to generalize. First, I failed to choose dependable knowledge from a noisy Internet. Then I failed to know what would make the language complete enough for real work. The names of Warren, Reed, Gabriel, Lenat, Gries, and Dijkstra do not rise automatically above a sea of plausible code [4]. I needed the expertise to recognize them—and still to judge the work rather than merely recognize the names.

## 4.4 Case 4: Difficulty of communication

I wrote a small Prolog program and asked all popular GenAI tools to rewrite it as English Horn clauses. This experiment was designed to check whether a GenAI can communicate effectively. The test is easy because Prolog conveys only a few concepts. I tested the following:

- Facts.
- Conditional clauses.
- Commitment.
- Functors that can represent verbs or adjectives.

Let us talk a little about functors. I will limit myself to a few functors. When I say **pretty(X)**, I mean that **X is pretty**; the functor is playing the role of an adjective. When I say **weights(N, 0.1)**, I mean that **N weights 0.1**.

Consider the program below. A conditional clause has a head, a neck:- (meaning if), and a conjunction of conditions. The head is true if the conjunction is true. For instance, the second clause reads — N weights 0.2 if N $>$ 999. An exclamation mark represents commitment. Then, the third clause reads — add(N, N, Acc) if Prolog commits$_{\text{to}}$ writeln(Acc), and therefore it gives up the next clause.

```
weights(N, 0.1) :- N < 1000, !.
weights(N, 0.2) :- N > 999.

add(N, N, Acc) :- !, writeln(Acc).
add(N, End, Partial) :-
    weights(N, W),
    Accumulator is Partial + W,
    Next is N + 1,
```

```
    add(Next, End, Accumulator).
```

Given the limited semantics that I described, I asked three GenAIs to write the above Horn clauses in English. All failed in the first attempt. Only one succeeded in the fourth attempt.

```
N weighs 0.1 if N < 1000, and prolog commits to it.

N weighs 0.2 if N > 999.

adding from N to N into Acc holds if
    prolog commits to writing Acc.

adding from N to End into Partial holds if N weighs W,
    Accumulator is Partial + W,
    Next is N+1,
    and adding from Next to End into Accumulator holds.
```

From this experiment, one can say that GenAIs have difficulty communicating information. In the case of a complex subject, we don't notice their communication failures, but the toy example of writing Prolog Horn clauses into English Horn clauses makes their limitation clear.

# 5 Discussion by GoofyAI

The three episodes expose distinct but related failure modes. In the quicksort case, the generated code solved separate representations rather than finding the shared indexed-store abstraction. In the plotting case, it presented a single fitted example as a general renderer. In the Prolog case, it optimized one requested behavior without establishing whether the resulting system implemented a useful subset of the language. These are failures of task modeling and validation, not merely syntax or execution.

A practical review protocol follows from the cases. First, state the required invariants and the class of inputs before requesting code. Second, test at least two structurally different instances so that hardcoded assumptions become visible. Third, inspect constants, branches, data-shape assumptions, and representation-specific operations. Fourth, maintain a capability checklist derived from intended use rather than from the most recent failing test. Finally, keep a domain expert responsible for deciding whether the abstraction and validation criteria are adequate. Passing tests remain necessary, but tests selected by the same narrow interpretation that produced the code can

provide false reassurance.

This account has limitations. It is a qualitative analysis of three episodes from one human–AI collaboration, not a controlled comparison of models or prompting strategies. The cases were selected because their artifacts survived and their errors were instructive. They therefore demonstrate failure mechanisms but do not estimate prevalence. Future work could encode the proposed review protocol in preregistered tasks and compare defect detection, generalization, and human effort across models and reviewer expertise levels.

# 6 Conclusion

People often rely on AI as an assistant in many tasks, such as coding, finance, healthcare and law. However, because GenAI can't sense the real world, its utility remains limited to information retrieval and processing. Even in these areas, it's still a good idea to have a knowledgeable professional in the loop, since when dealing with computers, a lack of contact with outside reality can create problems.

First, GenAIs can't generalize from examples, as the Kaplan-Meier plotter example revealed. Second, as the incomplete program reveals, GenAIs can't build exhaustive checklists. Finally, although a GenAI's output may resemble correct prose, that prose may not be rooted in solid information.

## 6.1 Conclusions by GoofyAI

Plausible and executable GenAI output can still be wrong at the level that matters most: the problem definition. The recurring remedy is disciplined human verification grounded in domain knowledge, explicit invariants, multiple representative cases, and a capability checklist. AI-assisted programming should therefore be evaluated not only by whether code runs, but by whether the implementation generalizes, preserves the intended semantics, and remains useful beyond the example that elicited it.

# 7 Generative AI Use Disclosure

Generative AI was both the subject of this case study and a contributor to the first-person reconstructions labeled as GoofyAI's accounts. The human authors selected the episodes, checked the surviving code and technical claims, determined the argument, and accept responsibility for the manuscript. This

disclosure should be revised by the authors if the journal's submission form requires the names, versions, dates, or specific uses of the AI systems.

# 8 References


1. Philippos Apolinario Costa, David T. Tse, and Pasquale Benedetto, "Neoadjuvant Intra-arterial Cytoreductive Chemotherapy Improves Outcomes in Lacrimal Gland Adenoid Cystic Carcinoma," *The Oncologist* 29, no. 3 (2024): 263–269.

2. Felipe F. Souza, Gina D'Amato, Emily Elizabeth Jonczak, Philippos Costa, Jonathan C. Trent, Andrew E. Rosenberg, Raphael Yechieli, H. Thomas Temple, Pradip Pattany, and Ty K. Subhawong, "MRI T2 Mapping Assessment of T2 Relaxation Time in Desmoid Tumors as a Quantitative Imaging Biomarker of Tumor Response: Preliminary Results," *Frontiers in Oncology* 13 (2023): 1286807.

3. John Darlington, "Program Transformation," *BYTE Magazine* 10, no. 8 (August 1985).

4. Robert B. Terwilliger, "Toward Tools to Support the Gries/Dijkstra Design Process," *ACM SIGSOFT Software Engineering Notes* (1993).